CONFIG & SECRETS

Від хардкоду до ротації без рестарту: env, zod, @nestjs/config, образи, секрет-менеджери

Заняття №11

Node.js Pro

Лектор
Кирило Орлов

r_d

СТЕК НА СЬОГОДНІ: ENV · DOTENV · ZOD · NEST-CONFIG · DOCKER · PG

Перша половина — конфігурація	Друга половина — секрети
хардкод — конфіг у коді, секрет у git	@nestjs/config — схема в DI + shutdown
process.env — 12-factor, але fail-late	docker-образ — де світиться секрет
dotenv — .env і контракт .env.example	ротація пароля БД без рестарту
zod — fail-fast: не стартуй зламаним	Секрет — не змінна, а керований ресурс

НАСКРІЗНА НИТКА

Крізь усю лекцію їде один пароль БД. Щоразу два питання: ХТО його бачить і КОЛИ ми дізнаємось, що конфіг зламаний. Обидві відповіді рухаємо: менше очей, раніша помилка.

ПЛАН ЗАНЯТТЯ



- Драбинка: хардкод → env → dotenv
- Zod-схема: fail-fast замість fail-late (демо A)
- Схема у @nestjs/config + shutdown
- Секрет в образі: шари, inspect, history (демо B)
- Ротація пароля БД без рестарту (демо C)
- Infisical наживо — сховище (демо D)
- Сід IaC на Pulumi (демо E)
- Менеджери, анти-патерни, межі, ДЗ

ШІСТЬ МІСЦЬ, ДЕ СВІТИТЬСЯ СЕКРЕТ

01. git-історія

git rm прибирає файл з HEAD, але не з історії — НАЗАВЖДИ

02. лог застосунку

крок 1 чесно друкує db=postgres://app:S3cr3t_…@… у stdout

03. шар образу

COPY . . без .dockerignore — і .env їде в шар: cat /app/.env

04. docker inspect

ENV з Dockerfile живе у конфігу образу — Config.Env видно всім

05. docker history

рядок ENV DB_PASSWORD=… видно у списку кроків збірки

06. CI-логи

echo $SECRET у пайплайні; маскування є не скрізь і не завжди

КРОК 1. КОНФІГ, ЗАШИТИЙ У КОД

const DB_URL = 'postgres://app:S3cr3t_...' прямо в сирці

інший порт чи база = міняти код і деплоїти заново

один білд не може поїхати у dev і prod одночасно

секрет у git, у лозі старту — і в кожному git clone

Пароль, який хоч раз потрапив у git, вважається скомпрометованим: git rm прибирає файл з HEAD, але не з історії комітів. Далі допомагає тільки ротація.

```
КРОК 1 НАЖИВО

$ node stages/01-hardcode.mjs
[step1] слухаю :21100;
db=postgres://app:S3cr3t_
   Hardcoded_2026@db.internal…
                  ↑
    секрет — у лозі старту

$ curl -s localhost:21100/
{"step":1,
 "dbHost":"db.internal:5432",
 "dbUser":"app"}

$ git log -p
+const DB_URL =
+  'postgres://app:S3cr3t_…'
                  ↑
  і в історії git — назавжди
```

КРОК 2. process.env: 12-FACTOR, ФАКТОР III

```
// stages/02-env-raw.mjs (скорочено)
// 12-factor, фактор III: конфіг — у середовищі
import http from 'node:http';

const port = Number(process.env.PORT ?? 21100);
const timeoutMs = Number(process.env.TIMEOUT_MS);
//   'abc' → NaN            забули → NaN, і байдуже

const server = http.createServer((req, res) => {
  const db = new URL(process.env.DB_URL);
  //   ↑ впаде ТУТ, на першому запиті, не на старті
  res.setHeader('content-type', 'application/json');
  res.end(JSON.stringify(
    { step: 2, dbHost: db.host, timeoutMs }));
});

server.listen(port, () => console.log(
  `[step2] слухаю :${port}; TIMEOUT_MS=${timeoutMs}`
  + ' (бачиш NaN? отож)'));

// $ env -u DB_URL -u TIMEOUT_MS node 02-env-raw.mjs
// $ curl -s -m 3 localhost:21100/   → curl exit=52
// TypeError: Invalid URL, input: 'undefined'
//   → процес мертвий: fail-late у чистому вигляді
```

Конфіг переїхав у середовище — гарантій не додалось

1 · той самий білд + інші змінні = dev / staging / prod

2 · process.env — Record<string, string | undefined>

3 · без DB_URL стартує «зеленим», падає на ПЕРШОМУ запиті

TIMEOUT_MS не задано → NaN — і жодної помилки

урок: fail-late — помилка конфігурації чекає на прод

«ЧЕК-ЛИСТ ПЕРЕД ЗЛЕТОМ»: FAIL-FAST ЯК АНАЛОГІЯ

ЧАСТИНА КОНФІГУРАЦІЇ

- env-змінні процесу
- zod-схема конфігурації
- fail-fast: exit=1 на старті
- fail-late (кроки 2–3)
- .env.example у git

АНАЛОГІЯ (АВІАЦІЯ)

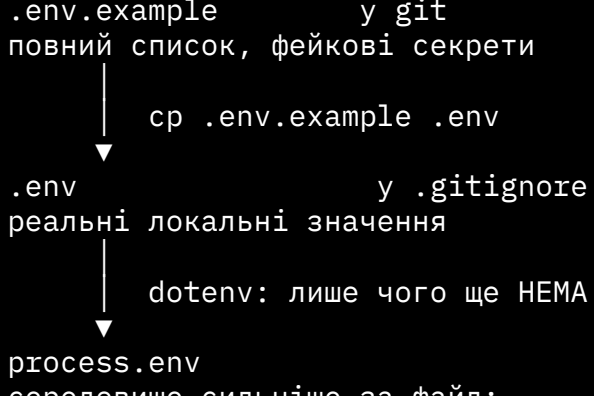
- перемикачі й прилади в кабіні
- чек-лист перед злетом
- проблема → лишаємось на землі
- дізнатись про неї вже в повітрі
- ламінований шаблон чек-листа

Схема, як чек-лист, перевіряє лише ФОРМУ. DB_URL може бути валідним URL мертвої бази — досяжність ловлять health checks, не zod.



КРОК 3. DOTENV: КОНВЕНЦІЯ .ENV / .ENV.EXAMPLE

- у dev 15 змінних — ніхто не експортує їх руками
- у проді змінні ставить платформа: compose, K8s, systemd
- dotenv кладе у process.env ТІЛЬКИ те, чого там ще нема
- додав змінну в код → додай у .env.example тим самим комітом
- типова помилка: класти реальний .env у git «щоб зручно»
- валідності досі НЕМАЄ: без DB_URL — знову крок 2



```
// stages/04-zod-failfast.mjs (скорочено) — або валідний конфіг, або стоп
// схему як інструмент бачили на L#4 (Fastify: 06 і 10-mini-fastify)
// там предмет — ЗАПИТ, на кожен виклик; тут — СЕРЕДОВИЩЕ, раз на старті
const EnvSchema = z.object({
  PORT: z.coerce.number().int().min(1).max(65535),   // env — рядки!
  DB_URL: z.url({ protocol: /^postgres$/ }),
  LOG_LEVEL: z.enum(['debug', 'info', 'warn', 'error']).default('info'),
  TIMEOUT_MS: z.coerce.number().int().positive().default(5000),
});

const parsed = EnvSchema.safeParse(process.env);
if (!parsed.success) {
  console.error('✗ Застосунок НЕ стартує — невалідна конфігурація:');
  for (const issue of parsed.error.issues)
    console.error(`    ${issue.path.join('.') || '(root)'}: ${issue.message}`);
  console.error('  Порівняй свій .env з .env.example.');
  process.exit(1);       // fail-fast: оркестратор не пустить у трафік
}
const config = Object.freeze(parsed.data);   // далі існує лише config
```



```
// stages/05-nest-config/src/env.schema.ts (скорочено)
// схема — ЄДИНЕ джерело правди: і рантайм-перевірка, і тип
export const envSchema = z.object({
  PORT: z.coerce.number().int().min(1).max(65535),
  DB_URL: z.url({ protocol: /^postgres$/ }),
  LOG_LEVEL: z.enum(['debug', 'info', 'warn', 'error']).default('info'),
  TIMEOUT_MS: z.coerce.number().int().positive().default(5000),
});

export type Env = z.infer<typeof envSchema>;   // тип виводиться сам

export function validate(raw: Record<string, unknown>): Env {
  const parsed = envSchema.safeParse(raw);
  if (!parsed.success) throw new Error(/* список зламаних змінних */);
  return parsed.data;    // Error → Nest не стартує взагалі
}
```



КРОК 5. @NESTJS/CONFIG: СХЕМА ЖИВЕ У VALIDATE()

ConfigModule.forRoot({ isGlobal: true, validate }) — і все

validate() виконується ОДИН раз — до створення DI-графа

зламаний env → ExceptionHandler, exit=1: застосунку немає

ConfigService<Env, true>: get('PORT', { infer: true }) → number, без undefined. Ендпоінт /config віддає dbHost і dbUser — пароль назовні не їде ніколи

МІСТОК: ЩО ТУТ СПРАВДІ НОВЕ
Контейнер, що рекурсивно збирає граф за токенами, ми писали руками на L#4 (крок 08-own-container), а forRoot() — звичайна функція, яка повертає опис модуля. Нове в цій лекції рівно одне: опція validate, тобто МОМЕНТ перевірки — до створення DI-графа, а не при першому get().



```
// stages/05-nest-config/src/main.ts (скорочено)
async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  app.enableShutdownHooks();    // ← закриваємось і виходимо САМІ
  process.on('SIGTERM',
    () => console.log('[step5] SIGTERM → закриваюсь чисто'));
  const config = app.get(ConfigService<Env, true>);
  await app.listen(config.get('PORT', { infer: true }));   // number
}
bootstrap();

// L#5 відповідав, чи сигнал ДОЛЕТИТЬ (exec- проти shell-форми CMD).
// Тут долітає — питання, що з ним робить процес. Три стани (заміряно):
//   слухача нема              → смерть миттєва, запит у польоті обірвано
//   слухач нічого не закриває → не вийде НІКОЛИ: docker stop 10 с → SIGKILL
//   enableShutdownHooks()     → закрився і вийшов сам   ← наш рядок
// $ DB_URL='mysql://oops' node dist/main.js
// ERROR [ExceptionHandler] Error: Невалідна конфігурація:
//   DB_URL: Invalid URL          → exit=1, Nest не стартував
// $ kill -TERM <pid> → [step5] SIGTERM → закриваюсь чисто
```

МОМЕНТ ВИЯВЛЕННЯ: ТРИ ТОЧКИ НА ШКАЛІ

Кроки 2–3

перший запит у проді: TypeError: Invalid URL — процес мертвий

Кроки 4–5

старт процесу: exit=1 і людський список зламаних змінних

check:env у CI

ще на PR: .env.example розійшовся зі схемою → exit=1

Чим лівіше по шкалі — тим дешевше
npm run check:env звіряє .env.example зі схемою в обидва боки (зразок у репо — scripts/check-env-example.mjs). Постав у CI — і «додав змінну, забув контракт» ловиться на PR, а не у колеги після git pull.

КОНФІГ ≠ СЕКРЕТ: ДВІ РІЗНІ ДИСЦИПЛІНИ

Аспект	Конфіг: PORT, LOG_LEVEL	Секрет: DB_PASSWORD, ключі
У git	так — .env.example, дефолти	ніколи, навіть «тимчасово»
Де живе	env-змінні процесу	файл-маунт або менеджер
Зміна	новий деплой — нормально	ротація, бажано без рестарту
Витік	дрібниця: порт і так видно	інцидент: міняти скрізь
Хто бачить	уся команда, логи, дашборди	процес у рантаймі — і все

КРОК 6. ОБРАЗ ПАМ'ЯТАЄ ВСЕ: ШАРИ І КОНФІГ

01. ENV DB_PASSWORD=... у Dockerfile живе в КОНФІГУ образу

02. inspect і history читають конфіг без запуску

03. COPY . . без .dockerignore — і .env їде в шар

04. RUN rm .env НЕ рятує: шари append-only

05. registry — це git для образів: docker pull бачить усе

ЩО ТУТ НОВЕ ПРОТИ L#5

З L#5 беремо готовим: шари append-only, .dockerignore вирішує, що їде у контекст. Нове — ДРУГИЙ канал: ENV живе не в шарі ФС, а в конфігу образу, і .dockerignore його не закриває взагалі. Обидва читаються БЕЗ запуску контейнера — досить прав на docker pull.

```
# stages/06-secrets-leak — Dockerfile.bad проти Dockerfile.good
# bad: обидві «зручності» лишаються в образі НАЗАВЖДИ
ENV DB_PASSWORD=Baked_In_Image_Pa55   # ← видно в docker inspect
COPY . .                              # ← .env їде В ШАР образу

# good: той самий COPY, але .dockerignore відсіює .env
COPY . .                              # секрет приїде в РАНТАЙМІ:
                                      # docker run -e … / K8s Secret

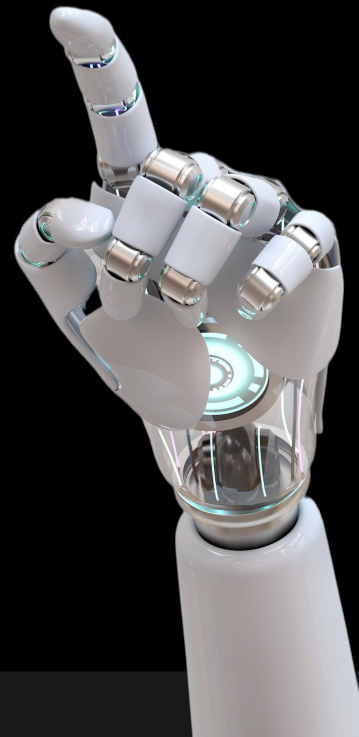
# $ npm run step6
# bad:  cat /app/.env  → DB_PASSWORD=Prod_Pa55w0rd_v1
# bad:  docker history → ENV DB_PASSWORD=Baked_In_Image_Pa55
# good: cat /app/.env  → No such file or directory
# good: docker inspect → без паролів
```



МАРШРУТ СЕКРЕТА: ЩОРАЗУ МЕНШЕ ОЧЕЙ

ДЕ ЖИВЕ ПАРОЛЬ	СКІЛЬКИ ЖИВЕ	ХТО ЙОГО БАЧИТЬ
у коді / git	вічно	всі клони, вся історія
↓		
в образі (шар, ENV)	роки	будь-хто з docker pull
↓		
env процесу	до рестарту	inspect, /proc, діти
↓		
файл-маунт	до ротації	процес із правами
↓		
сховище	до ротації	процес із машинною ідентичністю
↓		
dynamic (lease, TTL)	хвилини–години	один клієнт, сам згасає

кожен щабель нижче: менше копій · коротше життя · менше очей
демо B — стрибок з «образу», демо C — з env на файл
демо D (після перерви) — з файла у сховище



КРОК 7. РОТАЦІЯ БЕЗ РЕСТАРТУ: ДВА ПРИЙОМИ

прийом 1: секрет — ФАЙЛ, не env: env «замерзає» на старті

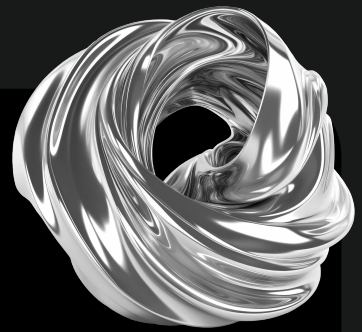
прийом 2: password у pg.Pool — ФУНКЦІЯ на кожен конекшн

старі з ′єднання живуть: пароль перевіряється на handshake

rotate.sh: ALTER ROLE → файл → pg_terminate_backend

pool.on('error') обов ′язковий: 57P01 не має вбити процес

вікно між ALTER і файлом: у проді — два користувачі, що чергуються (alternating users)



```
// stages/07-rotation/app.mjs (скорочено) — пул переживає ротацію
const SECRET_FILE = new URL('./secrets/db_password', import.meta.url);

const pool = new pg.Pool({
  host: '127.0.0.1', port: 21110,
  database: 'shop', user: 'app_user',
  password: async () =>                       // ← функція, не рядок:
    (await readFile(SECRET_FILE, 'utf8')).trim(), // на КОЖЕН конекшн
  max: 3,
});

// ротація вб'є idle-з'єднання → пул емітить 'error'; без обробника
// Node падає з Unhandled 'error' event:
pool.on('error', (e) => console.log(
  `[step7] сервер закрив idle-з'єднання (${e.code}) — пул відкриє нове`));

// $ npm run step7:rotate → /db відповідає далі, uptimeSec РОСТЕ:
// ДО → ALTER ROLE + файл + terminate → ПІСЛЯ: більший — той самий процес
```

КРОК 8. INFISICAL НАЖИВО

той самий код без сховища не стартує: DB_URL — undefined

infisical run кладе значення в env ДОЧІРНЬОГО процесу

--env prod: інші значення без ребілду і правок файлів

на диску значень НЕМАЄ: у .secrets/ — ключ ВІД сховища

env — знімок на момент exec: ротація тут НЕ безрестартна

ціна: новий SPOF і новий секрет — ENCRYPTION_KEY сховища

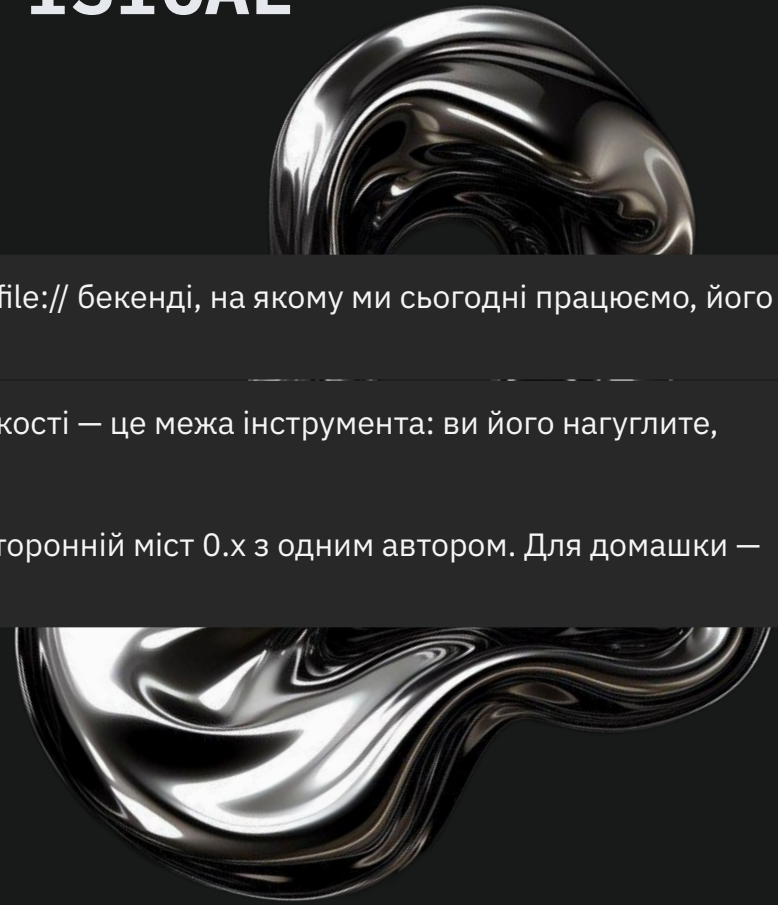
«ГОТЕЛЬНА КЛЮЧ-КАРТА»: DYNAMIC SECRETS

ЧАСТИНА СЕКРЕТІВ	АНАЛОГІЯ (ГОТЕЛЬ)
• dynamic secret під клієнта • TTL / lease — термін дії • revoke — миттєве відкликання • ротація master-креденшела • де ламається: менеджер упав…	• своя картка кожному гостю • діє рівно до дати виїзду • рецепція деактивує будь-коли • перепрошити замки, не картки • …рецепція злягла — ніхто не заселиться

МЕНЕДЖЕРИ СЕКРЕТІВ: КОРОТКА МАПА

Інструмент	Що це	Коли брати
Infisical	open-source менеджер, self-host у compose	основний менеджер курсу; у ДЗ обов ' язково
HashiCorp Vault	еталон: dynamic secrets, lease, аудит	ентерпрайз-масштаб
хмарні KMS/SM	AWS SM · GCP KMS · Azure KV: ротація + IAM	усе вже в цій хмарі
SOPS + age	шифровані секрети прямо в git, GitOps-флоу	малі команди, IaC
Doppler	SaaS-менеджер, сильний DX, синк у CI й хмари	не хочеш хостити сам

МЕЖА: PULUMI ESC ⇔ INFISICAL



Pulumi ESC працює ТІЛЬКИ з Pulumi Cloud. На локальному file:// бекенді, на якому ми сьогодні працюємо, його немає

Тому секрети в курсі веде Infisical, а не ESC. Це не оцінка якості — це межа інструмента: ви його нагуглите, тому кажу заздалегідь

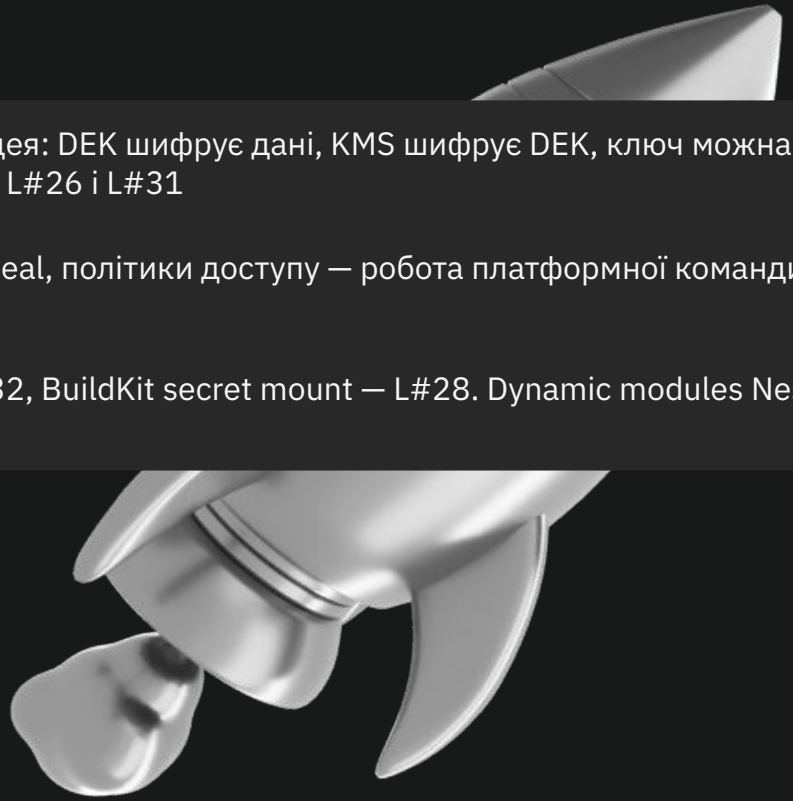
⚠ Офіційного Pulumi-провайдера для Infisical НЕ існує: є сторонній міст 0.x з одним автором. Для домашки — цікаво, у прод — ні

ЩО СВІДОМО ЛИШАЄМО ЗА ДУЖКАМИ СЬОГОДНІ

Envelope encryption і BYOK — сьогодні лише ідея: DEK шифрує дані, KMS шифрує DEK, ключ можна принести свій. Практика — у хмарному блоці, L#26 і L#31

Прод-розгортання Vault чи Infisical — HA, unseal, політики доступу — робота платформної команди, а не сьогоднішні 90 хвилин

⚠ K8s Secrets — L#28, секрети в CI/CD — L#32, BuildKit secret mount — L#28. Dynamic modules Nest глибоко — бонус-матеріали лекції



КРОК 9. Seed INFRASTRUCTURE AS CODE

СТАН

тека з json зв ′язує рядок декларації з реальним ресурсом: усередині — реальні id

ДРЕЙФ

хтось полагодив руками повз декларацію; refresh це бачить, up — лікує

ІДЕМПОТЕНТНІСТЬ

той самий up удруге → 3 unchanged: результат від різниці, а не від кількості запусків

Конфіг ми винесли з коду в декларацію — інфраструктуру виносять так само
І мовою, якою ви вже пишете: це TypeScript, не YAML. Сьогодні це seed, а не блок про IaC — повний блок на L#28 (Pulumi + Kubernetes), далі L#31 (AWS) і L#32 (pulumi preview у пул-реквесті)

ПІДСУМКИ + МІСТ ДО L#12

КОНФІГ

схема на старті: fail-fast, коерція, typed ConfigService

СЕКРЕТ

не в git, не в образі, не в env — файл або менеджер

РОТАЦІЯ

password-функція + pool.on('error') = зміна пароля без рестарту

ДЗ #11

5 балів · дедлайн 03.09 · zod-схема + check:env · секрети поза git і образом · hw-11 + PR · **сховище Infisical**: dev ≠ prod, infisical run, ротація · README «Configuration» · бонуси без балів: файл-ротація, Pulumi

ДАЛІ

L#12 PostgreSQL: той самий pg.Pool — тепер під EXPLAIN і індекси. Змінні наступних ДЗ — у цю ж схему

Q&A

???

r_d

ЗАВЖДИ Є КУДИ ЗРОСТАТИ